Projet RCP211
Intelligence artificielle avancée

Grossi Valério

17 Avril 2025
2024-2025 Semestre 1

Sommaire

1. Introduction **2**
2. Présentation de l'algorithme **2**
3. Présentation de l'étude **3**
3.1 Objectif 3
3.2 Problèmes traités 3
4. Mise en Oeuvre **6**
4.1 Environnement 6
4.2 Représentation latente des observations 7
4.3 Spécification des récompenses 7
4.4 Fonction de valeur d'états-actions et Politique 7
5. Hyperparamètres **8**
6. Résultats obtenus **9**
7. Conclusion **12**

1. Introduction

Dans le cadre du module *Intelligence Artificielle Avancée RCP211*, deux blocs que sont les modèles génératifs et l'apprentissage par renforcement sont étudiés de manière approfondie. Le projet choisi est « Génération de scénarios et renforcement » et a pour objectif de mettre en pratique les notions théoriques assimilées dans ces deux blocs.

Dans ce projet la structure algorithmique est imposée, il s'agit de « Reinforcement Learning with Imagined Goals » (RIG) proposée par Ashvin Nair et Vitchyr Pong dans leur papier *Visual Reinforcement Learning with Imagined Goals* (2018).

Une description rapide de l'algorithme RIG est d'abord effectuée, puis est présentée l'étude réalisée, son objectif et les problèmes traités. La mise en œuvre de l'étude est ensuite exposée, puis sont présentés les résultats obtenus.

2. Présentation de l'algorithme

Une des caractéristiques de cet algorithme « Reinforcement Learning with Imagined Goals » réside dans la possibilité de spécifier à l'inférence des objectifs divers. L'agent autonome entraîné avec RIG peut ainsi accomplir différentes tâches, sa politique étant conditionnée par l'objectif renseigné.

Pour cela, lors de l'apprentissage, l'agent a été entraîné sur un grand nombre d'objectifs. L'agent apprend une représentation latente des observations à partir initialement d'une collection d'états récoltés lors d'une phase d'exploration de l'environnement. Dans cet espace latent, l'agent échantillonne des objectifs dits "abstraits", encode les observations et calcule les distances des observations aux objectifs, utilisées pour spécifier les récompenses. L'agent apprend une politique permettant d'atteindre ces objectifs arbitraires sous cette représentation latente.

L'apprentissage des représentations latentes et de la politique s'effectue conjointement, à la fin de chaque épisode les nouveaux états visités sont utilisés dans une mise à jour de la représentation latente.

3. Présentation de l'étude

3.1 Objectif

Comme proposé dans le sujet « Génération de scénarios et renforcement », l'objectif de l'étude choisie est de générer des positions du jeu d'échecs et d'apprendre à l'agent à réaliser la configuration demandée. Les seules positions échiquéennes considérées sont celles Roi et Tour blanches contre Roi Noir, ainsi que Roi contre Roi.

L'objectif est donc d'apprendre une représentation latente de ces positions à l'aide d'un modèle génératif, puis d'apprendre à l'agent (les Blancs) à réaliser ces positions spécifiques.

Une contrainte majeure est rapidement apparue concernant l'utilisation de RIG (ou toute autre méthode) pour la réalisation par un agent de positions du jeu d'échecs. Dans le papier de Ashvin Nair et Vitchyr Pong, l'algorithme est utilisé sur des tâches robotiques, par exemple déplacer à l'aide d'un bras robotisé toute sortes d'objets différents à certaines positions. Contrairement au cadre d'application de RIG par Ashvin Nair et Vitchyr Pong, l'agent dans l'environnement échiquéen ne peut forcer une position spécifique sur l'échiquier.

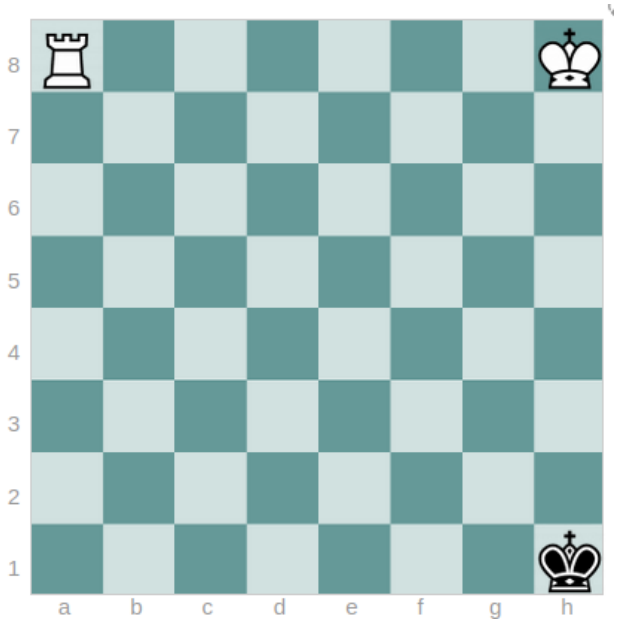
De cette observation, qui s'applique quel que soit l'environnement échiquéen considéré, j'ai décidé de restreindre drastiquement les positions du roi noir et de guider ses actions. J'ai ainsi considéré trois problèmes, laissant chacun plus ou moins de liberté au Roi Noir sur le choix de ses coups.

3.2 Problèmes traités

Problème 0:

Le Roi Noir débute l'épisode en h1 et n'effectue que des actions qui l'en rapproche (excepté bien sur lorsqu'il s'y trouve déjà, ne pouvant passer son tour). L'objectif pour l'agent est de déplacer ses pièces de sorte à atteindre la position Roi Blanc en h8, Tour Blanche en a8 et Roi Noir en h1.

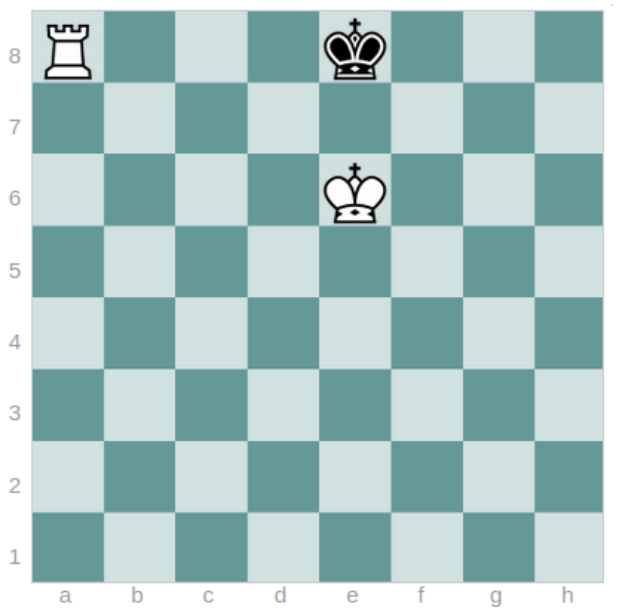
L'idée est que le Roi Noir reste globalement au même endroit de l'échiquier quel que soit les actions prises par l'agent et suffisamment éloigné des pièces de ce dernier afin que le problème puisse se ramener à un simple positionnement de pièces par l'agent sans considération d'influences extérieures à ce dernier.



Position Objectif Problème 0

<u>Problème 1</u>:

Le Roi Noir débute sur la huitième rangée et n'effectue que des actions pour y rester, il peut éventuellement descendre sur la septième rangée s'il n'a pas d'autres choix mais tente de revenir sur la huitième rangée dès que possible. Il balaie la dernière rangée vers la droite puis arrivé au bout vers la gauche et ainsi de suite. Cela a pour but de l'obliger à se rendre régulièrement sur la case e8 dont on voudra tester l'objectif de mat avec les blancs, dans la position Tour Blanche en a8, Roi Blanc en e6 et Roi Noir en e8.

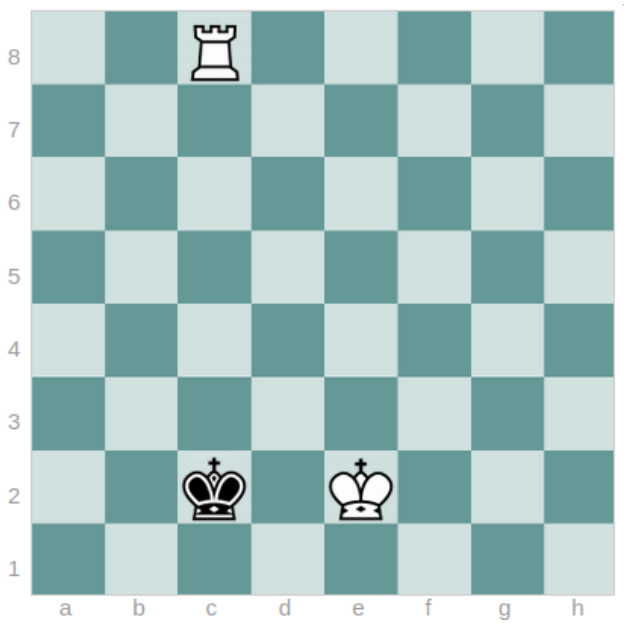


Position Objectif Problème 1

Problème 2:

Ici, on considère que le roi Noir est entièrement libre de ses actions tant qu'elles sont légales. Ses actions sont choisies aléatoirement. On souhaite apprendre à l'agent à mater le Roi Noir. Puisqu'on ne peut imposer de position spécifique, on moyenne dans l'espace latent sur toutes les positions de mat possibles (168 positions).

Bien qu'on moyenne sur l'espace latent je doute que cela fonctionne en théorie car je pense que cela peut certes amener le roi noir contre un bord mais n'incitera pas l'agent à placer ses pièces correctement vis à vis de la position du roi Noir pour mater. A défaut de pouvoir donc entraîner l'agent à mater il peut être intéressant de voir si l'agent apprend l'unique technique pour pouvoir forcer le roi adverse à reculer, à savoir l'opposition des rois suivi de l'échec avec la tour.



Objectif Problème 2 : Forcer le roi Noir contre un bord

L'agent a positionné son roi et sa tour de sorte à pouvoir faire reculer le roi noir, ici vers le bord gauche de l'échiquier.

4. Mise en Oeuvre

4.1 Environnement

Chaque observation représente une position sur l'échiquier et est définie par trois plans. La taille d'un plan est (8,8) et représente l'échiquier vu de haut. Chaque plan définit la présence ou non sur l'échiquier d'une des trois pièces considérées et leur position sur l'échiquier. On inscrit 1 dans la case correspondante si la pièce s'y trouve et 0 sinon. L'ordre des plans est présence roi Noir, roi Blanc puis Tour Blanche. Contrairement au dernier plan, les deux premiers contiennent toujours un 1 sur l'une de leur case.

On considère 2048 actions, ce qui correspond à toutes les actions possibles (en considérant l'information de la case de départ où s'exécute l'action) d'un roi et d'une tour sur l'échiquier.

L'épisode se termine lorsqu'un mat ou pat a lieu, au bout de 75 coups sans mater, ou lorsqu'il ne reste pas suffisamment de pièces pour mater c'est à dire lorsque la tour blanche est prise par le roi noir.

4.2 Représentation latente des observations

Les observations sont encodées dans un espace latent unidimensionnel comprenant 65 éléments. Les représentations latentes sont apprises grâce à un β-VAE.

Pour reconstruire les observations, on considère d'abord les 64 premiers éléments du code latent et on reconstruit la structure de taille (3,8,8) grâce à une série de convolutions transposées. On applique en sortie sur chaque plan reconstruit un softmax. Pour le dernier plan, on multiplie en plus chaque élément du plan par un gain obtenu en appliquant un sigmoid au 65 ème élément du code latent, censé caractériser la présence ou non de la tour blanche sur l'échiquier. On utilise comme loss de reconstruction du β-VAE la binary cross entropie.

Loss du β-VAE :

$$\mathcal{L}(x, \hat{x}, \mu, \log \sigma^2) = -\sum_{i=1}^{d} [x_i \log(\hat{x}_i + \epsilon) + (1 - x_i) \log(1 - \hat{x}_i + \epsilon)] + \beta \cdot \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + e^{\log \sigma_j^2} - 1 - \log \sigma_j^2 \right)$$

4.3 Spécification des récompenses

La récompense obtenue après l'exécution d'une action est la distance de Mahalanobis dans l'espace latent entre l'état encodé et l'objectif latent.

Récompense :

$$r_t = \sqrt{(z_{\text{next}} - z_g)^T \Sigma^{-1} (z_{\text{next}} - z_g)}$$

4.4 Fonction de valeur d'états-actions et Politique

La fonction de valeur d'états-actions ainsi que la politique sont paramétrées chacun par un réseau de neurones. On effectue des descentes de gradient stochastique par batch sur ces réseaux grâce à un replay buffer.

4.4.1 Fonction de valeur d'états-actions

Dans le papier de Ashvin Nair et Vitchyr Pong, les actions considérées sont les rotations des bras du robot. Ainsi, l'espace des actions considéré est continu et compatible du choix d'algorithme d'apprentissage par renforcement qui a été choisi, à savoir TD3.

Dans l'environnement défini pour notre problème, l'espace des actions étant discret, il n'a pas été possible de garder TD3. Néanmoins, l'approche consistant à estimer les valeurs d'états-actions via la minimisation de l'erreur de Bellman a été gardée.

Loss du réseau Critic :

$$\text{Q-loss} = \mathbb{E}_{z_g}\left[\mathbb{E}_{(z_t,a_t)}\left[\left(Q(z_t,a_t,z_g) - \left(r_t + \gamma \max_{a'} Q(z_{t+1},a',z_g)\right)\right)^2\right]\right]$$

Pour l'évaluation des valeurs d'états prochains - actions prochaines, on utilise un deuxième réseau critique « Target-Q-Network » , dont on aligne les paramètres à ceux du réseau critique tous les quatre épisodes.

4.4.2 Politique

Concernant le choix de la loss pour la politique paramétrée par un réseau neurones, on considère :

Loss du réseau Actor :

$$\text{Policy-loss} = -\mathbb{E}_{z_g}\left[\mathbb{E}_{(z_t,a_t)}\left[\log\left(\pi_\theta(a_t \mid z_t, z_g)\right) \cdot Q(z_t,a_t,z_g)\right]\right]$$

Un masque sur les actions légales est appliqué en sortie du réseau de la politique ainsi que lors du choix de la meilleure action prochaine pour le calcul de la Q-loss.

5. Hyperparamètres

Hyperparamètre	Valeur	Commentaire
vae_state_collection_size	20000	-
vae_state_collection_size_update	5000	-
mixture_coefficient	0.5	Ratio Buffer states / Exploration states
update_vae_every	10	Mise à jour du VAE tout les x épisodes
vae_num_epochs	200	-
vae_batch_size	100	-
vae_num_epochs_update	100	-
vae_batch_size_update	50	-
beta	0.5	Beta VAE
vae optimizer	Adam	-
vae learning rate	0.001	-
gamma	0.99	-
batch_size	100	Batch Size utilisé pour le calcul des coûts
actor optimizer	Adam	-
critic optimizer	Adam	-
target critic optimizer	Adam	-
actor learning rate	0.001	-
critic learning rate	0.001	-
target critic learning rate	0.001	-
update_target_every	4	Fréquence de mise à jour du réseau critic cible

6. Résultats obtenus

Dans le dossier contenant les codes figurent les vidéos de test de l'environnement pour les trois problèmes traités. Tous les scripts présents dans le dossier ont été créés pour ce projet.

Concernant la reconstruction par le VAE des observations encodées suite à l'entraînement de ce dernier, je n'ai pas obtenu les résultats espérés, les valeurs des éléments dans chaque plan étant égales. Voici un exemple représentatif de ce que j'obtiens, ci-dessous pour le problème 2 (liberté totale sur le positionnement des trois pièces):

Observation	**Reconstruction par le VAE**
tensor([[[0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [**1.**, 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.]]	tensor([[[0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156]]
[[0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., **1.**], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.]],	[[0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156]],
[[0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., **1.**, 0.], [0., 0., 0., 0., 0., 0., 0., 0.],	[[0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091], [0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091], [0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091], [0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091], [0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091], [0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091], [0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091],

[0., 0., 0., 0., 0., 0., 0., 0.]]])	[0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091, 0.0091]]],

On peut remarquer que pour le dernier plan, 0.0091 * 65 vaut environ 0.6 et cela n'est pas un hasard puisque le VAE a été entraîné sur des positions dont 60% contenaient une Tour blanche (dernier plan) et 40% n'en contenaient pas.

Voici un exemple représentatif de ce que j'obtiens pour le problème 0 (position initiale de la tour noir (premier plan) fixée):

Observation	**Reconstruction par le VAE**
tensor([[[0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., **1.**]],	tensor([[[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000], [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000], [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000], [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000], [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000], [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000], [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000], [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, **1.0000**]],
[[0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [**1.**, 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.]],	[[0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156], [0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156, 0.0156]],
[[0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.], [0., 0., 0., 0., 0., 0., 0., 0.],	[[0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079], [0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079], [0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079], [0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079], [0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079], [0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079], [0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079],

[0., 0., 0., 0., 0., 0., 0., 0.]]])	[0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079, 0.0079]]])

Là encore, je n'ai pas obtenu les résultats espérés, les valeurs des éléments dans chaque plan étant égales. On peut noter toutefois que le premier plan est lui bien reconstruit.

L'exigence sur la distribution des codes latents a bien été réalisée. En effet, des observations utilisées pour l'entraînement du VAE, on obtient bien des codes latents dont les variables sont centrées en zéro et d'écart-types faibles. Voici un exemple représentatif :

Fitted prior mu	**Fitted prior std**
tensor([[2.8626e-03, 3.9747e-03, 4.7555e-03, 8.1265e-04, 1.4645e-03, -7.2115e-04, -5.7930e-03, -1.4854e-02, 6.5746e-03, 6.6696e-03, -2.8559e-03, 4.6505e-03, 9.2562e-03, 6.7717e-03, -5.1571e-03, 1.5932e-03, -5.8256e-03, -1.1162e-02, -3.1625e-03, -3.0539e-03, 3.6398e-03, 3.0217e-03, 8.8566e-03, -6.4495e-03, 3.8727e-03, -7.5815e-03, 4.1867e-03, 1.1112e-03, 3.7650e-03, 6.8100e-03, 3.4717e-03, -5.2396e-04, -1.0093e-02, -2.0707e-03, 2.9561e-05, -2.9873e-03, -8.8934e-04, 3.7304e-03, -1.1040e-02, 5.9531e-03, 2.4936e-03, -4.3247e-03, -1.5102e-03, 5.7092e-03, 5.3385e-03, -4.1864e-03, 1.9914e-04, -9.3051e-03, -3.1280e-04, -3.8557e-03, 6.1017e-03, -1.9286e-03, -4.7994e-04, -2.1658e-03, 1.9142e-04, -4.3836e-03, -1.1175e-02, 8.3404e-03, 2.6088e-03, -4.5338e-03, -2.3213e-03, 8.0312e-03, 6.4720e-03, -1.2935e-02, 3.4999e-01]], grad_fn=<MeanBackward1>)	tensor([[0.0070, 0.0084, 0.0054, 0.0092, 0.0108, 0.0070, 0.0089, 0.0113, 0.0109, 0.0064, 0.0113, 0.0065, 0.0090, 0.0118, 0.0069, 0.0098, 0.0071, 0.0100, 0.0091, 0.0081, 0.0096, 0.7027, 0.0086, 0.0091, 0.0084, 0.0074, 0.0101, 0.0068, 0.0081, 0.0065, 0.0082, 0.0100, 0.0071, 0.0087, 0.0108, 0.0087, 0.0067, 0.0094, 0.7177, 0.0089, 0.0043, 0.0074, 0.0086, 0.0079, 0.0059, 0.0057, 0.0094, 0.0090, 0.0061, 0.0046, 0.0063, 0.0114, 0.0102, 0.0070, 0.0084, 0.0087, 0.0073, 0.0129, 0.0079, 0.0085, 0.0054, 0.0061, 0.0089, 0.7150, 0.3218]], grad_fn=<StdBackward0>)

En ce qui concerne l'entraînement de la fonction de valeur d'état-action et politique, j'obtiens des NaN sur les scores en sortie du réseau politique suite à seulement deux descentes de gradient de la fonction de valeur d'état-action paramétrée et de la politique. Les NaN apparaissent lors du passage dans les trois premières couches linéaires de la politique, qui d'un vecteur de taille deux fois l'espace latent renvoie les scores associées à toutes les actions référencées, utilisés ensuite pour le calcul des log-probabilités sur les actions légales à l'aide du masque des actions légales.

7. Conclusion

Bien que l'entraînement du VAE et de la politique, complexe, n’a pu aboutir, ce projet a été l’occasion de comprendre et mettre en pratique la méthode d’apprentissage par renforcement RIG.

Les deux mécanismes croisés de cette méthode, l’apprentissage d’une représentation latente et l'entraînement d’une politique compatible de divers objectifs latents, s'avèrent très utiles pour des applications au jeu d’échecs.

Utiliser un espace latent permet d’obtenir des objectifs latents réalisables, en moyennant sur un ensemble de positions dont individuellement on ne pourrait espérer les obtenir spécifiquement.

Utiliser une politique conditionnée par l’objectif permet d'entraîner un agent pour tout type de tâches souhaitables du jeu d’échecs, suivant la phase de jeu ; réussir son ouverture, prendre le centre, mettre son roi à l’abri, promouvoir ses pions, mater en finale.